



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Final Report of the Project

Ονοματεπώνυμο: ΙΑΣΩΝ ΚΑΡΑΠΡΟΔΡΟΜΙΔΗΣ
ΠΑΚΙΤΣΑΚ ΝΤΕΛΙΑ-ΜΑΡΙΑ
ΚΥΡΙΑΚΟΣ ΚΑΛΑΦΑΤΣΗΣ

Αριθμός Μητρώου: 1115202200064
1115202200125
1115202200058

Περιεχόμενα

1	Introduction	2
2	Late Materialization	2
2.1	Value_t Representation	2
2.2	Performance Improvement	3
3	Column-Store Execution	3
3.1	The column_t Structure	3
3.2	Changes in the Join Pipeline	4
3.3	Performance Improvement	4
4	Unchained Hashtable	5
4.1	Design and Layout	5
4.2	Parallel Build	5
4.3	Performance Improvement	5
5	Indexing optimization	5
5.1	Performance Improvement	6
6	Parallel Execution	6
6.1	Parallel Probe Execution	6
6.2	Parallel Scan Execution	6
6.3	Work Stealing	6
6.4	Time improvement	7
7	Performance Evaluation	8
7.1	Diagrams	8
8	Conclusion	8

1 Introduction

This project is based on the SIGMOD 2025 Programming Contest, which provides a ready-made join pipeline and a baseline implementation to experiment with. Instead of following a strict recipe the assignment encourages us to dive into the system, understand how it behaves and gradually improve its performance. Because of this experimental nature our focus is on practical optimization rather than purely theoretical analysis. We explore different data layouts, memory-efficient representations and faster join strategies always guided by real measurements. The goal is simple: take the given implementation and make it significantly faster while keeping the results correct. This report looks at the choices we made and the improvements we achieved along the way. All experiments were conducted on Iasonas's personal computer which has an AMD Ryzen-7 5700X with 8 cores and 16 threads and 16gb of DDR4 RAM running Ubuntu 24.04 and compiled with GCC 13.3.

2 Late Materialization

The baseline system relied on *early materialization*, where all attribute values were copied into a row-store representation before any join processing. This approach introduces substantial overhead particularly for `VARCHAR` attributes that never participate in join predicates. Late materialization defers the retrieval of non-join attributes until the final output stage reducing memory traffic and improving cache locality.

2.1 Value_t Representation

In the initial design the location of each value inside the column-store is represented using four separate fields of 16-bit variables:

1. table index (Tells us which table from the plan holds the desired value),
2. column index(The column that contains the data),
3. page index(Which page has the data inside the column.),
4. data index (Starting Position in the page)

Optimized Representation: Unified 64-bit value_t

When we pack all four pieces of information into one continuous 64-bit value instead of storing four separate 16-bit variables, the CPU can read everything it needs with a single memory access, which is much faster than loading four different fields from different memory locations. Modern processors are built to handle 64-bit values efficiently, so keeping the data together means fewer instructions, fewer cache misses, and less waiting for memory. More of these packed values fit inside the CPU cache at once so the join algorithm can process many more rows without slowing down. This compact layout also makes comparisons and checks much simpler because the CPU can work directly on one 64-bit number instead of combining several smaller ones. Overall using one packed 64-bit value makes the system faster, lighter, and much more friendly to how modern hardware actually works.

Pipeline Modifications

When we introduced the 64-bit `value_t`, the join pipeline became much more efficient because every operator now works with one compact value instead of four separate fields or a heavy variant type. During scanning, integers are stored directly and strings are represented only by small indexes so the system avoids copying large values and moves far less data through memory. This compact layout means the CPU can load all the needed information with a single instruction, which improves cache behavior and reduces the number of memory accesses during probing. Removing the old variant system also helped since the pipeline no longer performs type checks or branching just to understand what kind of value it is handling. As a result the join operators execute fewer instructions, touch less memory and keep more data in fast CPU caches, making the entire join process smoother, lighter, and noticeably faster.

2.2 Performance Improvement

This optimization reduced the total execution time from **149.000 ms** to **95.000 ms**, corresponding to a **36.24% improvement**.

The improvement is a result of several factors:

- **Avoidance of unnecessary string copies:** large `VARCHAR` values are materialized only once, at the final output stage
- **Reduced memory traffic:** compact 64-bit tuples minimize memory bandwidth usage
- **Improved cache locality:** more tuples fit in L1/L2 cache, accelerating probing.
- **Lower allocation overhead:** no intermediate string allocations occur during joins

3 Column-Store Execution

In the previous part we focused on late materialization meaning that instead of copying all column values upfront we kept only the integer keys needed for joins and fetched the remaining attributes only at the end if they were part of the final output. However, the join algorithm was still operating on a row-store structure. Even though the input tables were stored in a columnar format, the system converted them into rows, performed the join and then converted the results back into a columnar table. For more efficiency we will redesign the join algorithm so that all intermediate results are stored directly in a paged column-store format.

3.1 The `column_t` Structure

To support column-oriented intermediate results we created a new structure `column_t`, which represents a single column of a table. Using a column-store layout we improve performance because values belonging to the same attribute are stored contiguously inside pages allowing the CPU to iterate over them with fewer cache misses.

Each `column_t` contains:

- **num_rows:** the total number of values across all pages
- **type:** the data type of the column (`INT32` or `VARCHAR`)
- **pages:** a vector of pages where each page stores:

- the first 2 bytes: a `uint16_t` indicating how many values the page contains,
- the remaining bytes: a contiguous array of `value_t` entries.

This layout ensures that all values of a column are stored together, improving spatial locality and reducing memory traffic during joins.

3.2 Changes in the Join Pipeline

Significant modifications were required to make the join algorithm operate directly on column-store data.

The `scan_column_table()` function now receives a `ColumnarTable` and produces a `vector<column_t>`. One `column_t` is created per input column, and the function iterates through the pages of the input table. Also we have the `extract_keys_from_column()` function builds the hash table from the join-key column. It iterates through all pages, reads the number of rows, and scans the `value_t` entries. For each valid key, the function inserts the row index into the hash table, creating a new entry if needed. The complexity of this phase is $\mathcal{O}(n)$, where n is the number of rows on the build side.

Before probing the prefix arrays are constructed for every column. A prefix array maps a global row index to the correct page and local offset enabling constant-time navigation inside paged data. During probing the algorithm iterates through the pages of the probe-side column. For each `value_t` it reconstructs the join key and checks for matches in the hash table. If the key exists the algorithm retrieves all matching row indices from the build side and fetches the required values using `get_value()` which uses prefix arrays for fast lookup. Each result is appended to the output columns using `insert_value_to_page()`. The complexity of the probe phase is $\mathcal{O}(n \log P)$, where n is the number of probe-side rows and p is the total number of pages in every column.

Finally the `convert_column_t_to_columnar()` function transforms the vector of `column_t` objects into the final `ColumnarTable`. It initializes the output table with the correct row count and inserts values column by column.

3.3 Performance Improvement

Switching from row-store to column-store results in reduced execution time from **95.000 ms** (after late materialization) to **35.000 ms**, corresponding to a **76.51% improvement over the baseline** and a **63.15% improvement over the late-materialization version alone**.

The performance gains come from:

- **Column pruning:** only the necessary columns are stored and accessed
- **Sequential memory access:** contiguous values improve cache behavior
- **Reduced copying:** no full tuples are constructed during joins
- **Lower memory footprint:** fewer allocations and smaller intermediate structures

We chose a column-oriented representation because it stores values of the same attribute together in memory. This improves cache locality: the CPU can read consecutive values efficiently without jumping across unrelated data, which makes joins significantly faster. Using a vector of pages also avoids allocating memory for each individual value. Pages act as fixed-size blocks that store many entries at once, keeping data organized and making it easy to scale to large tables by simply adding new pages when needed.

4 Unchained Hashtable

4.1 Design and Layout

We implemented the Unchained Hashtable based on the specifications from the paper by Altan Birler, Tobias Schmidt, Philipp Fent, and Thomas Neumann on hash tables for join processing [1]. This hashtable was specifically designed for join processing; its most significant feature is the use of a Bloom filter to exclude impossible matches with $O(1)$ complexity. Given its low false-positive rate, this represents a massive improvement over the $O(\log n)$ complexity encountered during failed matches in other hashtable designs, such as Robin Hood hashing. Since this hashtable is specialized for joins, construction is divided into two phases: the insertion phase and the probe phase. This allows for rapid insertions and highly efficient probes due to the Bloom filter, with only a slight overhead during the finalization of the build process.

4.2 Parallel Build

While the probes were already thread-safe, the insertion and build phases required modifications to support concurrency. We needed a mechanism for threads to independently insert tuples into the hashtable while enabling a parallel build. The solution was partitioning: each thread adds items to its own local partition, and upon completion, partitions are assigned to threads for final processing. Each thread collects and processes its assigned partitions independently. This architecture required separating the build into two distinct stages: the preparation stage, which initializes the data structures for storing and probing tuples, and the migration stage, where tuples are moved from temporary storage to their final locations for probing. To address memory allocation bottlenecks, we replaced frequent `malloc` calls with a custom slab allocator. This allocator utilizes a three-level design to manage a large pre-allocated memory pool, minimizing system calls. We employed a linked list to manage memory chunks, ensuring that connecting partition chunks maintains $O(1)$ complexity, unlike a vector, which would require reallocating and moving pointers.

4.3 Performance Improvement

After replacing `std::unordered_map` with the Unchained Hashtable, the total runtime dropped from **35,000 ms** to **28,700 ms**. This improvement stems from the hashtable's ability to scale with high multiplicity and its efficiency during highly selective probes—a direct result of the Bloom filter. Following the implementation of the parallel build, the total runtime was further reduced to **27,000 ms**. While this initial gain seemed modest, further examination revealed that the probing phase consumes significantly more time than key extraction and the hashtable build combined. We chose for to have 8 threads and 32 partitions which after experiments was deemed the best setup.

5 Indexing optimization

The indexing optimization focuses on avoiding unnecessary copying during the scan phase. In the original implementation every column (including INT32 columns without NULL values) were fully copied into new pages inside each `column.t`. Copying is still required for VARCHAR columns and for INT32 columns that contain NULLs, because their layout depends on bitmaps, offsets, and string-continuation pages. However when an INT32 column has no NULL values the input pages already store fixed-width integers in a clean and contiguous format. By checking the page headers and the NULL bitmap the system can detect when a column has no missing values. In these cases,

the `column_t` is set up to reference the original input pages directly avoiding memory allocation. To support this zero-copy approach the `column_t` structure was extended with a `valid` flag that indicates whether the column owns its pages or simply indexes the input table. When `valid = false`, the scan phase skips page construction entirely and reuses the input page pointers. During materialization, the conversion function reads values directly from the original fixed-width buffers without checking bitmaps or reconstructing `value_t` entries. This optimization removes a large amount of copying and improves cache locality, especially for tables with many integer columns. If a column contains NULLs or is of type `VARCHAR` the system safely falls back to the original copying path. Overall, this produces a faster and lighter indexing process that reduces execution time while preserving correctness.

5.1 Performance Improvement

The indexing optimization further reduced execution time by eliminating unnecessary copying for INT32 columns that contain no NULL values. The execution time is reduced from **29.000 ms** (after late materialization) to **20.000 ms**, corresponding to a **86.6% improvement over the baseline**

The performance gains come from:

- **Zero-copy indexing:** INT32 columns without NULLs reuse input pages directly
- **Lower allocation overhead:** fewer page allocations and no temporary buffers
- **Faster scanning:** skipping bitmap checks and value reconstruction for clean integer columns

6 Parallel Execution

We used the STL thread library to parallelize the join pipeline.

6.1 Parallel Probe Execution

After measuring the total time the join pipeline spent in each phase of its execution with a Python script, we concluded that the probe phase was taking the most execution time, as it performs the bulk of the work. Fortunately, the page design of the `column_t` structure made parallelizing the probe phase very easy; it required no synchronization since each thread takes a single page and processes it independently.

6.2 Parallel Scan Execution

The scan already runs in parallel from our initial implementation (derived from the base solution, which also ran in parallel) with 12 threads. However, since the scan is memory-bound and spends most of its time waiting for I/O, running it in parallel versus serially made no significant difference in execution time. We decided to leave it as it was.

6.3 Work Stealing

To implement work stealing, we altered the parallel loops slightly to use an atomic variable to count the number of pages needing to be processed. This allowed threads to continue processing pages until none were left, ensuring each thread performed the maximum amount of work possible.

6.4 Time improvement

After making the scan, key extraction, and probe phases parallel, we saw an improvement from **20.000 ms** to **7.900 ms**—a massive improvement that cut the total runtime to less than half. After implementing work stealing, the runtime dropped further to **7.200 ms**. Most of this gain was achieved by making the probe parallel, as it had the most work to do and the parallelization efficiency was very high. We concluded that the best setup was to use all threads available on our machine for probing and the default 12 for the scan.

7 Performance Evaluation

7.1 Diagrams

Approach	Time (ms)	Improvement (from basic)
Basic Solution	149.000	—
Late Materialization	95.000	36.24%
Column Store	35.000	76.51%
Unchained Hashtable	28.000	81.21%
Indexing optimization	20.000	86.58%
Parallel Execution	7.900	94.70%
Work stealing	7.200	95.17%

Table 1: Performance comparison of optimization stages

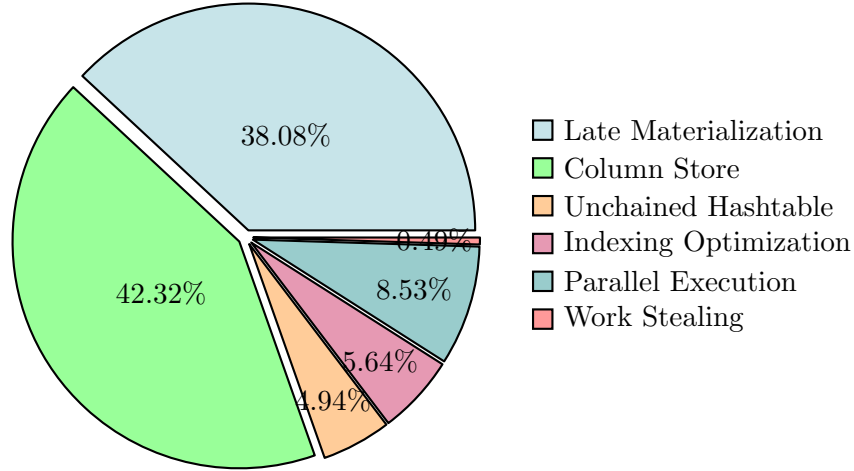


Figure 1: Contribution of each optimization to the total reduction in execution time

8 Conclusion

Through targeted optimizations—late materialization, column-store execution, unchained hashtable and parallelization—we achieved over 95% reduction in join execution time, with a final execution time of just over 7.000 ms compared to the starting 149.000 ms. These results highlight the importance of memory layout, cache locality, and efficient hashing in analytical query processing.

References

- [1] Altan Birler, Tobias Schmidt, Philipp Fent, and Thomas Neumann. Simple, Efficient, and Robust Hash Tables for Join Processing. 2025.